WIE

Posts & Reels

Complete API Documentation for React Native Developers

`Base URL: http://localhost:5010/api/post`

All routes require Bearer token • v1.0

Table of Contents

01 Service Overview & Base URL

The Post API runs on the WIE Media Service at port 5010. It manages photo and video posts (including carousels with up to 10 media items), a personalised feed, explore discovery, likes, comments, shares, saves, tags and post settings.

Service Config

```
Base URL (development) : http://localhost:5010/api/post
Base URL (Android emu) : http://10.0.2.2:5010/api/post
Route prefix: /api/post
Media is stored on Cloudinary. Max 10 files per carousel post.
All routes require a valid Bearer token.
```

Post Lifecycle

State	Meaning
`isDeleted=false`	Active post — visible according to visibility setting.
`isDeleted=true`	Soft-deleted. Hidden from all queries. Media removed from Cloudinary asynchronously.
`isArchived=true`	Archived by owner. Hidden from public feed but accessible to owner.
`isPinned=true`	Pinned post — returned first (sorted before createdAt) on user profile.

Visibility Values

Value	Access
`public`	Anyone can view, like, and comment (subject to block status).
`followers`	Only the owner's followers can view and interact.
`only_me`	Private — only the owner can view.

02 Authentication

Every endpoint requires a Bearer token obtained at login from the User Service. In React Native store the token in AsyncStorage and inject it via an Axios request interceptor.

TypeScript

```
// src/services/postService.ts — Axios instance
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';
const BASE = Config.MEDIA_API_URL || 'http://10.0.2.2:5010/api/post';
export const postApi = axios.create({ baseURL: BASE, timeout: 30000 });
postApi.interceptors.request.use(async cfg => {
const t = await AsyncStorage.getItem('token');
if (t) cfg.headers.Authorization = `Bearer ${t}`;
return cfg;
});
```

A 401 response means the token is expired or missing. Clear AsyncStorage and redirect to the Login screen.

03 Post Data Model

Every post returned from the API follows this shape. Understand it before building your UI components.

Post Object — Top-Level Fields

Field	Type	Required	Description
`_id`	`string`	—	Unique post ID (MongoDB ObjectId as string).
`userId`	`string`	—	Owner user ID.
`owner`	`object?`	—	Enriched owner info: { id, username, name, profile_picture, is_verified }
`mediaItems`	`MediaItem[]`	—	Ordered array of media (images/videos). Up to 10 items.
`caption`	`string?`	—	Post caption. Max 2200 characters.
`visibility`	`string`	—	public | followers | only_me
`locationLabel`	`string?`	—	Display name of the location sticker.
`locationPlaceId`	`string?`	—	Google Place ID.
`locationLat`	`number?`	—	Latitude (decimal degrees).
`locationLng`	`number?`	—	Longitude (decimal degrees).
`taggedUsers`	`Tag[]`	—	Users tagged in the post with optional x/y position.
`mentions`	`string[]`	—	User IDs @mentioned in the caption.
`likeCount`	`number`	—	Total number of likes (virtual).
`commentCount`	`number`	—	Total number of comments (virtual).
`shareCount`	`number`	—	Total share count.
`saveCount`	`number`	—	Total save count (virtual).
`hasLiked`	`boolean`	—	True if the current viewer has liked this post.
`hasSaved`	`boolean`	—	True if the current viewer has saved this post.
`commentsDisabled`	`boolean`	—	Owner has disabled comments.
`likesHidden`	`boolean`	—	Owner has hidden the like count from others.
`isPinned`	`boolean`	—	Post is pinned — appears first on user profile.
`isDeleted`	`boolean`	—	Soft-deleted flag (always false in API responses).
`createdAt`	`string`	—	ISO 8601 creation timestamp.
`updatedAt`	`string`	—	ISO 8601 last-update timestamp.

MediaItem Sub-Object (inside mediaItems[])

Field	Type	Required	Description
`url`	`string`	—	Cloudinary delivery URL.
`type`	`string`	—	image | video

width	number?	—	Width in pixels.
height	number?	—	Height in pixels.
duration	number?	—	Video duration in seconds.
aspectRatio	string	—	Computed ratio label: 1:1 | 4:5 | 1.91:1 | 16:9
order	number	—	Position in the carousel (0-based).
format	string?	—	File format (jpg, mp4, etc.).

TaggedUser Sub-Object (inside taggedUsers[])

Field	Type	Required	Description
userId	string	—	User ID of the tagged person.
x	number?	—	Horizontal position as % (0-100) of the image width.
y	number?	—	Vertical position as % (0-100) of the image height.
mediaIndex	number	—	Which carousel item the tag is on (0-based).

04 Create Post

Creates a new photo or video post. Accepts up to 10 media files in a single request (carousel). All files are uploaded to Cloudinary before the post document is saved. Mention and tag notifications are sent asynchronously after the response.

Method	Endpoint	Auth	Description
POST	/create	■ Yes	Create a post. multipart/form-data. Up to 10 media files.

Request — multipart/form-data Fields

Field	Type	Required	Description
media	File[]	Yes	1-10 image or video files. Field name must be 'media'.
caption	string	No	Post caption text. Max 2200 characters. Supports @userId mentions.
visibility	string	No	public | followers | only_me. Default: public
taggedUsers	JSON	No	Stringified JSON array of tag objects: [{ userId, x, y, mediaIndex }]
locationLabel	string	No	Display name for the location sticker.
locationPlaceId	string	No	Google Place ID.
locationLat	number	No	Latitude.
locationLng	number	No	Longitude.

Success Response (HTTP 201)

JSON Response


```
{
"success": true,
"data": {
"_id": "post_abc123",
"userId": "user_me",
"mediaItems": [
{
"url": "https://res.cloudinary.com/.../photo.jpg",
"type": "image",
"width": 1080,
"height": 1350,
"aspectRatio": "4:5",
"order": 0
}
],
"caption": "Great day at the beach!",
"visibility": "public",
"likeCount": 0,
"commentCount": 0,
"createdAt": "2025-12-01T10:00:00.000Z"
}
}
```


React Native — Upload Single or Multiple Images

TypeScript

```
import { launchImageLibrary } from 'react-native-image-picker';
const createPost = async () => {
// Allow multi-select for carousel (up to 10)
const result = await launchImageLibrary({
mediaType: 'mixed',
selectionLimit: 10,
quality: 0.85,
});
if (!result.assets?.length) return;
const formData = new FormData();
// React Native: use { uri, name, type } — NOT File/Blob
result.assets.forEach(asset => {
formData.append('media', {
uri: asset.uri,
name: asset.fileName || 'photo.jpg',
type: asset.type || 'image/jpeg',
} as any);
});
formData.append('caption', 'Great day at the beach!');
formData.append('visibility', 'public');
// Optional: location
formData.append('locationLabel', 'Bondi Beach');
formData.append('locationPlaceId', 'ChIJm7gp...');
formData.append('locationLat', '-33.8915');
formData.append('locationLng', '151.2767');
// Optional: tag people
formData.append('taggedUsers', JSON.stringify([
{ userId: 'user_abc', x: 45, y: 30, mediaIndex: 0 }
]));
const res = await postApi.post('/create', formData, {
headers: { 'Content-Type': 'multipart/form-data' },
onUploadProgress: e =>
setProgress(Math.round((e.loaded * 100) / (e.total || 1))),
});
return res.data.data; // Post object
};
```

Aspect Ratio Logic

Ratio	Condition
16:9	width/height > 1.7 (landscape video/photo)
1.91:1	width/height > 1.2 (Facebook-style landscape)
1:1	width/height > 0.9 (square)
4:5	width/height <= 0.9 (portrait — recommended for feed)

05 Post Feed

Returns a personalised feed of posts from users the viewer follows, plus the viewer's own posts. Pinned posts appear first for each author. Results are sorted newest-first after pinned.

Method	Endpoint	Auth	Description
GET	/feed	■ Yes	Personalised feed: own posts + following posts, newest first.

Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Posts per page. Max 50. Default: 20.

Response

JSON Response


```
{
"success": true,
"data": [
{
"_id": "post_abc",
"owner": {
"id": "user_xyz",
"username": "janedoe",
"name": "Jane Doe",
"profile_picture": "https://cdn.../avatar.jpg",
"is_verified": false
},
"mediaItems": [ /* MediaItem[] */ ],
"caption": "Beautiful sunset",
"likeCount": 142,
"commentCount": 18,
"shareCount": 5,
"hasLiked": false,
"hasSaved": false,
"isPinned": false,
"visibility": "public",
"createdAt": "2025-12-01T08:00:00.000Z"
}
],
"pagination": {
"page": 1,
"limit": 20,
"total": 84,
"hasMore": true
}
}
```

06 Explore Posts

Returns all public posts from public accounts, excluding the viewer's own posts. Use this to power a discovery or explore screen.

Method	Endpoint	Auth	Description
GET	/explore	■ Yes	All public posts from public accounts. Excludes own posts.

Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Posts per page. Max 50. Default: 20.

Response

JSON Response

```
{
"success": true,
"data": [ /* Post[] — public posts from public accounts */ ],
"pagination": { "page": 1, "limit": 20, "total": 540, "hasMore": true }
}
```

Private account posts never appear in Explore, even if marked visibility='public'. The server filters them out automatically.

07 Get User Posts

Returns posts for any user. Respects the account privacy gate — private accounts return 403 if the viewer does not follow them. Own profile always works. Pinned posts appear first.

Method	Endpoint	Auth	Description
GET	/user/:userId	■ Yes	Paginated posts for a user. Privacy-gated for private accounts.

URL Parameter

Item	Description
:userId	User ID of the profile being viewed.

Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Posts per page. Max 50. Default: 12.

Success Response (HTTP 200)

JSON Response

```
{
"success": true,
"data": [ /* Post[] with hasLiked, hasSaved, likeCount, commentCount, owner */ ],
"pagination": { "page": 1, "limit": 12, "total": 48, "hasMore": true }
}
```

Error Codes

```
403 This account is private — Viewer does not follow this private account
```

08 Get Single Post

Returns full post details including enriched owner info, viewer-specific flags (hasLiked, hasSaved), and visibility enforcement. Use this when navigating to a post detail screen.

Method	Endpoint	Auth	Description
GET	/:postId	■ Yes	Get a single post. Enforces visibility and follower-only gates.

Success Response (HTTP 200)

JSON Response

```
{
"success": true,
"data": {
"_id": "post_abc",
"owner": { /* user info */ },
"mediaItems": [ /* MediaItem[] */ ],
"caption": "Sunset vibes",
"likeCount": 142,
"commentCount": 18,
"hasLiked": true,
"hasSaved": false,
"visibility": "public",
"commentsDisabled": false,
"likesHidden": false
}
}
```

Error Codes

```
403 This post is private — visibility='only_me', viewer is not owner
403 Follow this user to see their posts — visibility='followers', viewer does not follow
404 Post not found
```

09 Update Post

Allows the post owner to edit the caption, visibility, and location details. Media files cannot be changed after creation. All fields are optional — only sent fields are updated.

Method	Endpoint	Auth	Description
PATCH	`/:postId`	■ Yes	Update caption, visibility, or location. Owner only.

Request Body — application/json

Field	Type	Required	Description
`caption`	`string`	No	New caption text. Max 2200 characters.
`visibility`	`string`	No	New visibility: public | followers | only_me
`locationLabel`	`string`	No	New location label.
`locationPlaceId`	`string`	No	New Google Place ID.
`locationLat`	`number`	No	New latitude.
`locationLng`	`number`	No	New longitude.

Success Response (HTTP 200)

JSON Response

```
{ "success": true, "data": { /* Updated Post object */ } }
```

10 Delete Post

Soft-deletes a post. The post is marked isDeleted=true and excluded from all queries. Media is removed from Cloudinary asynchronously after the response. Only the post owner can delete.

Method	Endpoint	Auth	Description
DELETE	`/:postId`	Yes	Soft-delete a post and remove media from Cloudinary. Owner only.

Success Response (HTTP 200)

JSON Response

```
{ "success": true, "message": "Post deleted" }
```

Show a confirmation dialog before calling this endpoint — deletion cannot be undone from the client side.

11 Post Settings

Allows the owner to toggle per-post settings without changing the content. Use this for the post options sheet (menu).

Method	Endpoint	Auth	Description
PATCH	/:postId/settings	■ Yes	Toggle comments, likes visibility, pin state. Owner only.

Request Body — application/json

Field	Type	Required	Description
commentsDisabled	boolean	No	true = disable comments. false = enable. Default: false.
likesHidden	boolean	No	true = hide like count from others. Default: false.
visibility	string	No	Change visibility: public | followers | only_me
isPinned	boolean	No	true = pin this post at top of profile. Default: false.

Success Response (HTTP 200)

JSON Response

```
{
"success": true,
"data": {
"commentsDisabled": true,
"likesHidden": false,
"visibility": "public",
"isPinned": false
}
}
```

Settings UI Examples

TypeScript

```
// Disable comments on a post
await postApi.patch(`/${postId}/settings`, { commentsDisabled: true });
// Pin a post to the top of profile
await postApi.patch(`/${postId}/settings`, { isPinned: true });
// Hide like count from public
await postApi.patch(`/${postId}/settings`, { likesHidden: true });
// Change to followers-only
await postApi.patch(`/${postId}/settings`, { visibility: 'followers' });
```

12 Likes & Reactions

A single endpoint toggles like/unlike with an optional emoji reaction. The server enforces a cooldown to prevent notification spam.

Method	Endpoint	Auth	Description
POST	`/:postId/like`	■ Yes	Toggle like / unlike. Optional emoji reaction. Sends notification.
GET	`/:postId/likes`	■ Yes	Get likes. Non-owner: count + hasLiked. Owner: full enriched list.

POST /:postId/like — Request Body

Field	Type	Required	Description
`emoji`	`string`	No	Emoji reaction. Default: heart. Any valid Unicode emoji.

POST /:postId/like — Response

JSON Response

```
// On like:
{ "success": true, "liked": true, "likeCount": 143 }
// On unlike:
{ "success": true, "liked": false, "likeCount": 142 }
```

GET /:postId/likes — Response (non-owner)

JSON Response

```
{ "success": true, "total": 142, "hasLiked": true }
// If owner has hidden likes:
{ "success": true, "total": null, "hidden": true }
```

GET /:postId/likes — Response (owner — full list)

JSON Response

```
{
"success": true,
"total": 142,
"hasLiked": false,
"likes": [
{
"userId": "user_abc",
"emoji": "heart",
"name": "Jane Doe",
"username": "janedoe",
"profile_picture": "https://cdn.../avatar.jpg",
"createdAt": "2025-12-01T11:00:00.000Z"
}
]
}
```

Supported Emoji Reactions

Emoji	Meaning
`heart`	Like / Love (default)
`fire`	Fire — wow
`laugh`	Funny
`wow`	Surprised
`clap`	Clap / Applause
`rocket`	Rocket — impressive

13 Comments — Add, List, Delete

Comments are stored inside the post document. Each comment can have nested replies. Adding a comment triggers a notification to the post owner (not for self-comments). Comments can be disabled by the post owner via post settings.

Method	Endpoint	Auth	Description
POST	/:postId/comments	■ Yes	Add a comment. Sends notification to post owner.
GET	/:postId/comments	■ Yes	Get paginated comments with replies and like counts.
DELETE	/:postId/comments/:commentId	■ Yes	Delete a comment. Post owner OR comment author can delete.

POST /:postId/comments — Request Body

Field	Type	Required	Description
text	string	Yes	Comment text. Max 500 characters.

POST /:postId/comments — Success Response (HTTP 201)

JSON Response

```
{
"success": true,
"comment": {
"userId": "user_me",
"text": "Stunning shot!",
"likes": [],
"replies": [],
"createdAt": "2025-12-01T11:30:00.000Z",
"name": "Jane Doe",
"profile_picture": "https://cdn.../avatar.jpg"
}
}
```

GET /:postId/comments — Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Comments per page. Max 100. Default: 20.

GET /:postId/comments — Response

JSON Response

```
{
"success": true,
"total": 18,
"comments": [
{
"_id": "comment_001",
"userId": "user_abc",
"text": "Stunning shot!",
"likeCount": 4,
"likes": ["user_xyz", "user_def"],
"createdAt": "2025-12-01T11:00:00.000Z",
"name": "Jane Doe",
"username": "janedoe",
"profile_picture": "https://cdn.../avatar.jpg",
"replies": [
{
"_id": "reply_001",
"userId": "user_xyz",
"text": "Totally agree!",
"likeCount": 1,
"createdAt": "...",
"name": "John Smith",
"username": "johnsmith"
}
]
}
],
"pagination": { "page": 1, "limit": 20, "hasMore": false }
}
```


Error Codes

```
403 Comments are disabled for this post — commentsDisabled=true, viewer is not owner
```

14 Comment Replies

Add a reply to an existing comment. Replies are nested inside comments (not paginated separately). The comment author receives a notification. Replies are returned inside comments[] in the GET comments response.

Method	Endpoint	Auth	Description
POST	/:postId/comments/:commentId/reply	■ Yes	Add a reply to a comment. Notifies comment author.

Request Body — application/json

Field	Type	Required	Description
text	string	Yes	Reply text. Max 300 characters.

Success Response (HTTP 201)

JSON Response

```
{
"success": true,
"reply": {
"userId": "user_me",
"text": "Totally agree!",
"likes": [],
"createdAt": "2025-12-01T11:45:00.000Z"
}
}
```

15 Like Comments & Replies

Toggle likes on comments and nested replies. Both are toggle endpoints — calling them again removes the like. Liking a comment sends a notification to the comment author (not for self-likes).

Method	Endpoint	Auth	Description
POST	`/:postId/comments/:commentId/like`	■ Yes	Toggle like on a comment. Notifies comment author.
POST	`/:postId/comments/:commentId/replies/:replyId/like`	■ Yes	Toggle like on a comment reply.

Response — both endpoints

JSON Response

```
// On like: { "success": true, "liked": true }
// On unlike: { "success": true, "liked": false }
```

16 Share Post

Shares a post to one or more users as a chat message (DM). The share is sent via the Chat Service gRPC client. The post's shareCount is incremented and the post owner receives a notification.

Method	Endpoint	Auth	Description
POST	/:postId/share	Yes	Share post to user DMs. Body: { receiverIds: string[] }

Request Body — application/json

Field	Type	Required	Description
receiverIds	string[]	Yes	Array of user IDs to share the post to via DM.

Success Response (HTTP 200)

JSON Response

```
{
"success": true,
"message": "Post shared",
"results": [
{ "receiverId": "user_abc", "chatId": "chat_001", "messageId": "msg_001" },
{ "receiverId": "user_def", "error": "Failed to send" }
]
}
```

The share is delivered as a special 'post_share' message type in the recipient's DM thread. The recipient sees a post preview card in their chat.

17 Save / Bookmark (Toggle)

Toggles save (bookmark) on a post. Calling the endpoint when the post is already saved will unsave it. An optional collection name can be provided to organise saved posts into named collections.

Method	Endpoint	Auth	Description
POST	`/:postId/save`	■ Yes	Toggle save/unsave. Body: { collection?: string }

Request Body — application/json

Field	Type	Required	Description
`collection`	`string`	No	Optional collection name (e.g. 'Travel', 'Recipes'). Default: no collection.

Response

JSON Response

```
// On save: { "success": true, "saved": true, "saveCount": 15 }
// On unsave: { "success": true, "saved": false, "saveCount": 14 }
```

18 Saved Posts

Returns all posts that the authenticated user has saved/bookmarked, sorted by most recently saved. Used for the Saved tab on the self-profile screen.

Method	Endpoint	Auth	Description
GET	/saved	■ Yes	Get all posts saved by the current user (paginated).

Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Posts per page. Max 50. Default: 12.

Response

JSON Response

```
{
"success": true,
"data": [ /* Post[] — the user's saved posts */ ],
"pagination": { "page": 1, "limit": 12, "total": 7, "hasMore": false }
}
```

19 Tag People

The post owner can tag other users in images with optional x/y coordinates (as percentages of image dimensions). Tags show as floating name pills on the image. Tagged users receive a mention notification. Blocked users cannot be tagged.

Method	Endpoint	Auth	Description
POST	/:postId/tags	■ Yes	Set tagged users on a post. Replaces existing tags. Owner only.
GET	/:postId/tags	■ Yes	Get all tagged users with enriched profile info and positions.

POST /:postId/tags — Request Body

Field	Type	Required	Description
taggedUsers	Tag[]	Yes	Array of tag objects. Replaces the entire existing tags list.

Tag Object Fields

Field	Type	Required	Description
userId	string	Yes	User ID to tag.
x	number	No	Horizontal position as % (0-100). 50 = centre.
y	number	No	Vertical position as % (0-100). 50 = centre.
mediaIndex	number	No	Carousel position this tag belongs to (0-based). Default: 0.

POST /:postId/tags — Example Body

JSON Body

```
{
"taggedUsers": [
{ "userId": "user_abc", "x": 45, "y": 30, "mediaIndex": 0 },
{ "userId": "user_def", "x": 70, "y": 60, "mediaIndex": 1 }
]
}
```

GET /:postId/tags — Response

JSON Response

```
{
"success": true,
"taggedUsers": [
{
"userId": "user_abc",
"x": 45,
"y": 30,
"mediaIndex": 0,
"name": "Jane Doe",
"username": "janedoe",
"profile_picture": "https://cdn.../avatar.jpg"
}
]
}
```

20 Full Route Reference

All routes are relative to http://localhost:5010/api/post. All require Bearer token. Static routes come before parameterised routes (:postId) in the router.

Static Routes (must be before /:postId)

Method	Endpoint	Auth	Description
GET	/feed	■ Yes	Personalised post feed
GET	/explore	■ Yes	Public discovery / explore posts
GET	/saved	■ Yes	Current user's saved posts
POST	/create	■ Yes	Create post (multipart — up to 10 media files)
GET	/user/:userId	■ Yes	Posts for a specific user (privacy-gated)

Parameterised Routes (/:postId)

Method	Endpoint	Auth	Description
GET	/:postId	■ Yes	Single post with visibility enforcement
PATCH	/:postId	■ Yes	Update caption / visibility / location
DELETE	/:postId	■ Yes	Soft-delete post
PATCH	/:postId/settings	■ Yes	Toggle commentsDisabled, likesHidden, isPinned
POST	/:postId/like	■ Yes	Toggle like/unlike with emoji
GET	/:postId/likes	■ Yes	Get likes (full list for owner, count for others)
POST	/:postId/comments	■ Yes	Add a comment
GET	/:postId/comments	■ Yes	Paginated comments with replies
POST	/:postId/comments/:commentId/reply	■ Yes	Add a reply to a comment
POST	/:postId/comments/:commentId/like	■ Yes	Toggle like on a comment
POST	/:postId/comments/:commentId/replies/:replyId/like	■ Yes	Toggle like on a reply
DELETE	/:postId/comments/:commentId	■ Yes	Delete a comment (post owner or comment author)
POST	/:postId/share	■ Yes	Share to user DMs
POST	/:postId/save	■ Yes	Toggle save/bookmark
POST	/:postId/tags	■ Yes	Set tagged users (replaces existing)
GET	/:postId/tags	■ Yes	Get tagged users with positions

21 React Native Integration Examples

Complete postService.ts

TypeScript

```
// src/services/postService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';
const BASE = (Config.MEDIA_API_URL || 'http://10.0.2.2:5010/api') + '/post';
export const postApi = axios.create({ baseURL: BASE, timeout: 30000 });
postApi.interceptors.request.use(async cfg => {
const t = await AsyncStorage.getItem('token');
if (t) cfg.headers.Authorization = `Bearer ${t}`;
return cfg;
});
// Feed & Discovery
export const getPostFeed = (page = 1, limit = 20) =>
postApi.get('/feed', { params: { page, limit } }).then(r => r.data);
export const getExplorePosts = (page = 1) =>
postApi.get('/explore', { params: { page } }).then(r => r.data);
export const getUserPosts = (userId: string, page = 1, limit = 12) =>
postApi.get(`/user/${userId}`, { params: { page, limit } }).then(r => r.data);
export const getSavedPosts = (page = 1) =>
postApi.get('/saved', { params: { page } }).then(r => r.data);
export const getPostById = (postId: string) =>
postApi.get(`/${postId}`).then(r => r.data.data);
// Interactions
export const toggleLike = (postId: string, emoji = 'heart') =>
postApi.post(`/${postId}/like`, { emoji }).then(r => r.data);
export const toggleSave = (postId: string, collection?: string) =>
postApi.post(`/${postId}/save`, { collection }).then(r => r.data);
export const sharePost = (postId: string, receiverIds: string[]) =>
postApi.post(`/${postId}/share`, { receiverIds }).then(r => r.data);
// Comments
export const getComments = (postId: string, page = 1) =>
postApi.get(`/${postId}/comments`, { params: { page } }).then(r => r.data);
export const addComment = (postId: string, text: string) =>
postApi.post(`/${postId}/comments`, { text }).then(r => r.data);
export const replyToComment = (postId: string, commentId: string, text: string) =>
postApi.post(`/${postId}/comments/${commentId}/reply`, { text }).then(r => r.data);
export const likeComment = (postId: string, commentId: string) =>
postApi.post(`/${postId}/comments/${commentId}/like`).then(r => r.data);
export const deleteComment = (postId: string, commentId: string) =>
postApi.delete(`/${postId}/comments/${commentId}`).then(r => r.data);
// Settings
export const updateSettings = (postId: string, patch: Record) =>
postApi.patch(`/${postId}/settings`, patch).then(r => r.data);
export const deletePost = (postId: string) =>
postApi.delete(`/${postId}`).then(r => r.data);
```

Post Feed Screen — Infinite Scroll

TypeScript

```
const [posts, setPosts] = useState([]);
const [page, setPage] = useState(1);
const [hasMore, setHasMore] = useState(true);

const loadFeed = async (nextPage = 1) => {
const res = await getPostFeed(nextPage, 20);
if (nextPage === 1) setPosts(res.data);
else setPosts(prev => [...prev, ...res.data]);
setHasMore(res.pagination.hasMore);
setPage(nextPage);
};

// FlatList with onEndReached:
<FlatList
data={posts}
keyExtractor={p => p._id}
renderItem={({ item }) => <PostCard post={item} />}
onEndReached={() => hasMore && loadFeed(page + 1)}
onEndReachedThreshold={0.5}
/>
```

Post Card — Like & Save Buttons (Optimistic Update)

TypeScript

```
// Optimistic update pattern
const handleLike = async (post: Post) => {
// Optimistic: flip state immediately
const newLiked = !post.hasLiked;
const newCount = newLiked ? post.likeCount + 1 : post.likeCount - 1;
updatePostInState(post._id, { hasLiked: newLiked, likeCount: newCount });

try {
const res = await toggleLike(post._id);
// Sync with server count
updatePostInState(post._id, {
hasLiked: res.liked,
likeCount: res.likeCount,
});
} catch {
// Roll back on error
updatePostInState(post._id, { hasLiked: post.hasLiked, likeCount: post.likeCount });
}
};
```

WIE Posts & Reels API • React Native Documentation • v1.0

Base: http://localhost:5010/api/post • All routes require Bearer token